

Performance budgets

Real, measured numbers behind acceptance criteria that would otherwise be unfalsifiable prose ("large worktrees stay responsive," "the app feels fast"). Each entry states what was measured, on what, and a concrete number a future change can be measured against — and can fail. Add a section per endpoint/ subsystem as they get their own criterion; do not fold this into `docs/RELEASE_GATES.md` (which is about which checks gate a release, a different question) or `docs/GIT_CLIENT_ROADMAP.md` (a milestone roadmap, not a measurement log).

Caveat that applies to every number in this file, stated once here rather than repeated per section: these are single runs on one development host (4 cores, this box, debug/unoptimized build unless a section says otherwise), not a statistically controlled benchmark suite — no warm-up runs, no repeated trials, no variance reported. Treat every number here as "real and reproducible," not "precise to the millisecond." A regression test derived from a budget uses a generous multiple of the measured number specifically because of this — see each section's own regression-test note.

`GET /api/status/v2` (#68e, M2.15)

What was measured. The real handler seam (`worktree_status_v2_for_repo` in `crates/git-vista-server/src/handlers/ read.rs`) end to end: the `git status --porcelain=v2 --branch -z spawn`, the `-z porcelain` read through the capped primitive, `parse_porcelain_v2_z` (#68b), and the full generation derivation (`read_generation_inputs`'s HEAD + every ref + index walk, plus the sha256 digest of the read bytes, #68c). Not just the git spawn alone — the generation derivation walks every ref in the repository too, and isolating only the parse would have missed that as a real cost center.

Reproduce with:

```
cargo test -p git-vista-server --bin git-vista-server -- --ignored --nocapture \
  large_worktree_responsiveness_ladder
cargo test -p git-vista-server --bin git-vista-server -- --ignored --nocapture \
  large_worktree_cap_boundary_in_file_count
```

(Both `#[ignore]` d — real file generation and repeated git spawns, no place in every cargo test/CI run. See their doc comments in `read.rs`.)

The ladder — worktree size vs. wall-clock time

Untracked files (`generate_untracked_files`), debug build, one run each, this host:

files changed	elapsed	cap hit?
100	4.413 ms	no
1,000	8.963 ms	no
5,000	17.800 ms	no
20,000	64.864 ms	no

Roughly linear in file count across two orders of magnitude — no evidence of a superlinear cost center (e.g. an accidentally quadratic ref walk or parse step) at these sizes. Untracked entries are the cheapest real record shape (no mode/hash fields to compute per entry, unlike a staged or unstaged change), so this ladder is closer to a best case than a worst case for a given file count — a worktree with the same file count but every file modified (carrying real mode/hash fields per porcelain record) would cost somewhat more per file, not measured separately here.

Where the 8 MiB read cap actually bites, in file count

Task 13 (#68c) chose an 8 MiB cap (`STATUS_V2_STDOUT_CAP`) and made a cap hit a 413, not a best-effort parse. Nobody had measured what worktree size that actually corresponds to. Measured: **450,000 untracked files with uniform 15-character names** (`bench-NNNNNN.txt, ? + name + NUL = 20 bytes/record ⇒ ~8.6 MiB`) reliably exceeds the cap — confirmed by `large_worktree_cap_boundary_in_file_count` asserting the call is refused, not corrupted-parsed. Total wall time for that run (dominated by creating 450,000 real files, not by `git status` itself) was **72.16 s**.

This is a lower bound for this specific naming scheme, not a universal constant.

A real worktree's actual paths are typically longer (directory nesting, real filenames) than a 15-character uniform name, and every non-untracked record (staged, unstaged, renamed, conflicted) carries mode and hash fields the untracked shape doesn't — both push the real-world file count that trips the cap down from 450,000, not up. Treat 450,000 as "the cap does not bite for any worktree size anyone has actually filed an issue about," not as "this is exactly where real users will hit it."

Stated budget

A `GET /api/status/v2` request against a worktree with up to 20,000 changed files must complete in well under 1 second on hardware comparable to this host. The measured 64.9 ms at 20,000 files leaves roughly 15x headroom before that budget — deliberately generous, not a tight fit to today's number, so the regression test below doesn't flake on a loaded CI runner while still catching a real regression.

Regression test: `worktree_status_v2_budget_holds_at_1k_files` in `read.rs` (not `#[ignore]` d — runs in every `cargo test`) asserts 1,000 changed files complete inside **2 seconds** — roughly 220x the measured 8.96 ms at that size, chosen loose enough to tolerate a slow/loaded runner while still catching an actual regression (e.g. the generation derivation's ref walk becoming accidentally quadratic, which would show up as a multi-second stall at a mere 1,000 files, nowhere close to this budget).

Diff per-render walks — `selectable_hunks` and `parse_unified_diff + flatten` (one component of #211, M2.16f)

Status: this is one component of the diff view's total cost, not a substitute for the windowed-render measurement. This section used to budget `hunk_nav`, the raw-text walk every diff render paid for labels and navigation targets. #361 deleted it — rendering and labels are now driven by the structured parser (`git_vista_protocol::diff::parse_unified_diff` flattened into `rows::DiffRows`) — but the cost center did not vanish with the function, it split in two, and BOTH successors are budgeted here:

- **the structured walk** (`parse_unified_diff + rows::flatten`), run once per patch by the detail panel, the full-screen viewer, and the staging view's spoken labels;
- **`selectable_hunks`** (`features/diff/core.rs`), the one surviving raw-text walk, run per staging render for selection anchors — kept on purpose because the staging view addresses hunks by line index into the raw text it renders (#215).

The failure mode is unchanged from the old section: "a 5 MB refactor diff can carry tens of thousands of hunks, and a rescan-per-hunk would stall the iPad's main thread before first paint." Neither walk has that shape today (all passes are $O(n)$); this budget exists so it's caught immediately if a future edit reintroduces it in either one.

What this does NOT cover — stated explicitly, not left implicit:

- The actual DOM/`<pre>` construction in `detail.rs/viewer.rs/staging_view.rs`. All are `#[cfg(target_arch = "wasm32")]`-gated; `cargo test --workspace` never compiles them and this repo has no wasm test harness. Unmeasured, and no test here claims otherwise.
- The windowing math itself (`row_heights`, `CumulativeHeights`, `render_window`, `scroll_to_reveal`) — that is measured separately, see the section below.
- Real-world patch shapes (renames, binary markers, combined merge headers, uneven hunk sizes). The generator produces one synthetic file with uniformly-sized hunks — the cheapest-per-byte shape, deliberately mirroring 68e's uniform-untracked-file generator — so this is closer to a best case than a worst case.

Reproduce with:

```
cargo test -p git-vista -- --ignored --nocapture diff_walk_ladder
```

(`#[ignore]` d — generates up to a ~7 MB synthetic patch, no place in every `cargo test/CI` run. See its doc comment in `features/diff/core.rs`.)

The ladder — hunk count vs. wall-clock time

Synthetic uniform hunks (`generate_patch`, 3 add/remove line pairs each), debug build, one run each, this host (2026-08-16, the run recorded when the ladder was retargeted from `hunk_nav`):

hunks	selectable_hunks	parse + flatten	patch bytes
100	0.398 ms	0.578 ms	14,037
1,000	3.128 ms	5.671 ms	141,837
2,000	6.487 ms	11.359 ms	285,837
10,000	31.635 ms	56.753 ms	1,437,837
20,000	64.686 ms	112.721 ms	2,897,837
50,000	162.808 ms	291.857 ms	7,277,837

Both roughly linear (2,000 → 20,000 hunks, a 10x increase, cost ~10x more time in each column) — no evidence of a superlinear cost center at these sizes. The structured walk costs about 1.7x the raw walk — the price of building owned `Strings` per line instead of scanning borrowed text — and stays comfortably inside the budget. The 2,000-hunk row sits just past `DIFF_PATCH_CAP` (200,000 bytes, the panel's patch cap in `handlers/read.rs`) — a realistic "hit the panel's cap" size, not an arbitrary round number. 50,000 hunks (~7 MB) sits past `DIFF_PATCH_CAP_FULL` (5,000,000 bytes, the full-screen viewer's cap) — the server would already have truncated a real patch this large before either walk ever saw it; it's included to see the shape holds past both caps, not because a real request reaches it.

Stated budget

Each walk, over a patch with up to 20,000 hunks (comparable to `DIFF_PATCH_CAP_FULL`), must complete in well under 1 second on hardware comparable to this host. The measured 112.7 ms for the slower (structured) walk at 20,000 hunks leaves roughly 8x headroom before that budget.

Regression tests, both in `features/diff/core.rs` (not `#[ignore]` d — run in every `cargo test`), each asserting over BOTH walks:

- `diff_walk_budgets_hold_at_2k_hunks` asserts 2,000 hunks (the panel-cap-realistic size above) complete inside **500 ms** per walk — roughly 44x the measured 11.4 ms of the slower one at that size.
- `diff_walks_scale_roughly_linearly_not_quadratically` asserts the 20,000-hunk run costs less than **25x** the 2,000-hunk run's time. This is the test that actually catches the regression named above: an accidental reintroduction of a rescan-per-hunk shape would show up as roughly a further 10x slowdown on top of the expected 10x (i.e. close to 100x for this 10x size increase) — the wall-clock budget alone would not reliably catch that until it got much worse, since 500 ms has a lot of headroom. Both tests also assert each walk found exactly as many hunks as the generator produced, so a fast-but-wrong answer (an early return, or a desynced countdown eating the rest of the patch) fails the test instead of passing it by accident.

Windowed diff render (#69c wired in, closing #211, M2.16f)

Status: this is the real measurement #211 asked for, now that the target exists.
PR #351 (merged 2026-08-07) wired `crates/git-vista-core::virtualize's`

`CumulativeHeights/visible_range` (#69c — 9 tests, previously zero consumers) into `crates/git-vista/src/detail.rs`'s render path: it now imports `line_heights`, `render_window`, `scroll_to_reveal`, `LineWrap` and `CumulativeHeights`, and builds a real window in its render closure (~line 635). Before that PR, "the virtualized diff view" named in

211 did not exist anywhere in the tree — this repo's prior session recorded

that gap honestly in the per-render walk section above (then budgeting `hunk_nav`) rather than measuring a substitute and calling it done. That gap is now closed.

What the windowed path actually is (read from source, not assumed):

- `line_heights(patch, line_height, wrap) -> Vec<f64>` (`crates/git-vista/src/features/diff/core.rs`) — per-line pixel heights. `LineWrap::Never` (the detail panel; `.detail-diff` is `white-space: pre`) gives one row per line. `LineWrap::Wrapped { columns }` (the full-screen viewer; `.viewer-pre` is `white-space: pre-wrap`) gives `ceil(chars/columns)` rows per line — **the full-screen viewer is not currently windowed**, deliberately; see "What this does NOT cover" below.
- `CumulativeHeights::new(&[f64])` (`crates/git-vista-core/src/virtualize.rs`) — $O(n)$ prefix sums over those heights, built once per patch change.
- `render_window(&CumulativeHeights, viewport_height, scroll_offset, overscan) -> RenderWindow { start, end, pad_top, pad_bottom }` — $O(\log n)$ per scroll query via binary search over the prefix sums.
- `scroll_to_reveal(&heights, index, viewport_height, current_scroll) -> Option<f64>` — used to bring a focused line into view.
- `crates/git-vista/src/detail.rs: DIFF_LINE_PX = 18.1, DIFF_OVERSCAN = 20;`
`accessible_patch_window(patch, focus, scope, window, reveal)` renders only the window's slice, each line keyed on its own index into the full patch (not the window's local index) so DOM identity survives scrolling.

What is covered here, and what is deliberately not:

- **Covered, host-testable, measured by cargo test:** the pure windowing math — `line_heights`, `CumulativeHeights::new`, `render_window`, `scroll_to_reveal` — over patches of stated hunk/line counts, at the panel viewport size. This is the $O(n)$ build + $O(\log n)$ query cost the primitive was designed to have; the numbers below confirm it holds at real sizes, not just in the 9 unit tests #69c shipped.
- **NOT covered — the wasm/DOM boundary.** Building the actual `<pre>` window in the browser (`accessible_patch_window`'s DOM construction, `detail.rs/viewer.rs`) is `#[cfg(target_arch = "wasm32")]`-gated. `cargo test --workspace` never compiles that code and this repo has no wasm test harness. **Paint time, layout thrash, and actual first-paint latency on a real device are not measured anywhere in this document.** A reader must not come away believing this budget covers what the user sees on screen — it covers the arithmetic that decides which lines get drawn, not the drawing.

- **NOT covered — the full-screen viewer.** `viewer.rs`'s `LineWrap::Wrapped` path is not windowed at all yet (confirmed by reading `viewer.rs`, not assumed); this budget is `panel-path (LineWrap::Never)` only.
- **Still separately covered — the per-render walks.** `selectable_hunks` and `parse_unified_diff + flatten` (see above) are upstream of and independent from this windowing math; they are not re-measured here.

Reproduce with:

```
cargo test -p git-vista --bin git-vista-ui -- --ignored --nocapture
virtualize_ladder
```

(#[ignore] d — it builds patches up to 50,000 lines, no place in every `cargo test`/CI run.)

The ladder — patch size vs. windowing cost

Synthetic patches, debug build, one run each, this host. `LineWrap::Never` is the panel's real mode (`.detail-diff` is `white-space: pre`); `Wrapped{80}` is measured so a number exists for whoever wires the viewer, which is **not** windowed today.

patch lines	wrap	CumulativeHeights::new	render_window query	lines rendered
1,000	Never	0.017 ms	0.0019 ms	86
1,000	Wrapped{80}	0.016 ms	0.0010 ms	86
10,000	Never	0.163 ms	0.0014 ms	86
10,000	Wrapped{80}	0.196 ms	0.0015 ms	86
50,000	Never	0.908 ms	0.0024 ms	86
50,000	Wrapped{80}	0.972 ms	0.0028 ms	86

The rendered-lines column is the result that matters. It is exactly 86 at every size, in both wrap modes, across a 50x range — because $\text{ceil}(800\text{px} / 18.1\text{px}) = 45$ lines fit the viewport and `DIFF_OVERSCAN = 20` adds 20 each side. That total is a function of viewport height and line height **only**. It does not grow with the patch, which is the entire property virtualization exists to provide. A window that grew with `n` would mean windowing had been silently bypassed.

The build cost scales linearly as documented: 50x the lines cost ~54x the time (16.75 μs → 908.08 μs), with no superlinear cost center visible. The per-scroll query stayed in a 1–3 μs band across the whole range, with no growth trend distinguishable from noise at microsecond scale — consistent with the documented $O(\log n)$ `partition_point` binary search, and effectively free against a 16 ms 60fps frame budget.

Where the caps sit. The generator produces ~66.8 bytes/line, so the 1,000-line row (~66.8 KB) is under `DIFF_PATCH_CAP` (200,000 bytes, the panel's cap); the 10,000-line row (~688 KB) is already **past** it. The panel — the only windowed surface today — could never receive the 10k or 50k patches, because the server truncates first. Its realistic

ceiling for this line shape is roughly **3,000 lines**. Those larger rows exist to show the shape holds well past both caps, not because a real request reaches them.

Stated budget

A diff of any size the panel or viewer can hold must satisfy three bounds on hardware comparable to this host:

1. **The once-per-patch `CumulativeHeights::new` build completes in well under 50 ms.** Measured 0.91 ms at 50,000 lines — roughly **55x headroom**.
2. **Each `render_window` query completes in well under one 16 ms frame.** Measured 2.4 μ s at 50,000 lines — roughly **6,000x headroom**. This is the number that decides whether scrolling is smooth.
3. **The rendered window stays bounded regardless of patch size.** Measured a constant 86 lines from 1,000 to 50,000.

Regression tests, both in `features/diff/core.rs`, neither `#[ignore]` d:

- `virtualize_query_budget_holds_at_50k_lines` — asserts the query completes under 5 ms, the build under 50 ms, and the window stays under 200 lines at 50,000 lines.
- `render_window_size_has_a_floor_and_does_not_grow_with_the_patch` — asserts the window has a **floor** as well as a ceiling.

Why two tests, and why the second is not redundant. The wall-clock bounds alone would not catch the regression that matters. A naive implementation that abandoned windowing and returned the full range would still pass the 5 ms query bound at 50,000 lines — 2.4 μ s has three orders of magnitude to spare — but fails the bounded-window assertion immediately.

Both were mutation-proven, and the second exists because the first was found insufficient: an "always return an empty window" mutation leaves the ceiling-only assertion **green**, since zero is under 200. Verified by applying both mutations (always-empty, always-full-range) to `render_window`'s body and confirming red, then restoring and reconfirming 30/30 green. A window that renders nothing is as broken as one that renders everything, and only the floor catches it.